Compiler Construction (CS-310)

Spring 2K26

Project task # 1

		Obt Marks
Std Name with Reg #		
Std Name with Reg #		
Std Name with Reg #		

Submission Note:

1. Submit complete running code (in zip folder) and output screens in a word file at Project link.
2. Cut-copy-paste from the internet or copied from your friend can lead to negative marking.
3. Individual marking will be done based on the knowledge and understanding of the code.
4. Attach this page as the front page of your assignment.
5. **Interface:** Webpage running on local host.
6. **Deadline:** **27 April 2026**

Introduction

A scanner is a program that takes a sequence of characters (the source file of the program) and produces a sequence of tokens that will be used to feed the compiler's parser. At the end of this step, the program developed (the very first part of our compiler) should be able to open a program's source file written in the ***W++ Language*** and recognize its tokens. In this assignment, you will build a **Token Analyzer** for the W++ programming language. Your program will take W++ source code as input, tokenize it, and generate a comprehensive statistical report including token frequency, line-wise distribution, and various metrics (see below).

W++ Language Quick Reference

Data Types: int, float, double, char, string
Operators: =, +, -, *, /, <, >, <=, >=, ==, !=, &&, ||
Separators: ;, ,, (,), {, }
Keywords: int, float, double, char, string, if, else, while, for, return, print, read, true, false
Literals

- **Integer:** 42, -17, 0, 1000
- **Float:** 3.14, -0.5, 10.0
- **Char:** 'A', 'z', '5', '\n', '\t'
- **String:** "Hello", "W++", "Line1\nLine2"

Identifiers

- Letters, digits, underscore, must start with letter or underscore
- Examples: x, counter, _temp, var123

Comments

- Single line: // This is a comment
- Multi-line: /* This is a multi-line comment */

Token Analyzer

Implement a **W++ Token Analyzer** that reads a W++ source file and produces a detailed statistical report.

Required Features

Your program must:

1. **Read W++ source code** from a file (.wpp extension)
2. **Tokenize** the code according to W++ language specification
3. **Generate the following statistical summary:**

a) **Token Type Summary Table** showing:

- Token Category (e.g., KEYWORD, IDENTIFIER, LITERAL, OPERATOR, SEPARATOR)
- Token Type (specific type like KEYWORD_INT, OPERATOR_PLUS, etc.)
- Quantity (total count of this token type)
- Percentage of total tokens
- Line Numbers (comma-separated list of lines where this token appears)

b) **Line-wise Token Distribution** showing:

- Line Number
- Total tokens on that line

c) **Identifier Statistics** showing:

- List of all unique identifiers
- Frequency of each identifier
- Lines where each identifier appears

d) **Literal Statistics** showing:

- List of all literals with their types
- Frequency of each literal value
- Lines where each literal appears

e) **Overall Summary** showing:

- Total number of tokens
- Total number of unique token types
- Total number of lines with code (excluding empty lines)
- Most frequent token type
- Least frequent token type
- Average tokens per line

Output Screen Sample

```
================================================================================
                    W++ TOKEN ANALYZER - STATISTICAL REPORT
================================================================================

INPUT FILE: sample.wpp
TOTAL LINES: 15
LINES WITH CODE: 12
TOTAL TOKENS: 87

================================================================================
1. TOKEN TYPE SUMMARY
================================================================================

Category         | Token Type              | Qty | %    | Lines
-----------------|-------------------------|-----|------|----------------------
KEYWORD          | KEYWORD_INT             | 4   | 4.60%| 1,3,5,8
KEYWORD          | KEYWORD_FLOAT           | 2   | 2.30%| 2,6
KEYWORD          | KEYWORD_IF              | 1   | 1.15%| 4
KEYWORD          | KEYWORD_ELSE            | 1   | 1.15%| 6
KEYWORD          | KEYWORD_WHILE           | 1   | 1.15%| 10
KEYWORD          | KEYWORD_PRINT           | 3   | 3.45%| 4,7,11
IDENTIFIER       | IDENTIFIER              | 15  | 17.24%| 1,2,3,4,5,6,8,9,10,11
LITERAL          | LITERAL_INTEGER         | 8   | 9.20%| 1,3,5,8,10
LITERAL          | LITERAL_FLOAT           | 3   | 3.45%| 2,6,9
LITERAL          | LITERAL_STRING          | 2   | 2.30%| 7,11
LITERAL          | LITERAL_CHAR            | 1   | 1.15%| 3
OPERATOR         | OPERATOR_ASSIGN         | 6   | 6.90%| 1,2,3,5,8,9
OPERATOR         | OPERATOR_PLUS           | 4   | 4.60%| 4,8,10,11
OPERATOR         | OPERATOR_MINUS          | 2   | 2.30%| 5,9
OPERATOR         | OPERATOR_MULT           | 2   | 2.30%| 4,8
OPERATOR         | OPERATOR_DIV            | 1   | 1.15%| 9
OPERATOR         | OPERATOR_LT             | 1   | 1.15%| 10
OPERATOR         | OPERATOR_GT             | 1   | 1.15%| 4
SEPARATOR        | SEPARATOR_SEMICOLON     | 12  | 13.79%| 1,2,3,4,5,6,7,8,9,10,11
SEPARATOR        | SEPARATOR_COMMA         | 4   | 4.60%| 1,2,5,8
SEPARATOR        | SEPARATOR_LPAREN        | 4   | 4.60%| 4,6,8,10
SEPARATOR        | SEPARATOR_RPAREN        | 4   | 4.60%| 4,6,8,10
SEPARATOR        | SEPARATOR_LBRACE        | 3   | 3.45%| 4,6,10
SEPARATOR        | SEPARATOR_RBRACE        | 3   | 3.45%| 5,7,11
COMMENT          | COMMENT                 | 0   | 0.00%| (ignored)
WHITESPACE       | WHITESPACE              | 0   | 0.00%| (ignored)

--------------------------------------------------------------------------------
```

```
3. IDENTIFIER STATISTICS
================================================================================

Identifier         | Frequency | Lines
-------------------|-----------|----------------------
x                  | 5         | 1,3,4,5,8
y                  | 3         | 1,3,6
result             | 4         | 2,4,7,9
counter            | 6         | 5,8,9,10,11,12
temp               | 2         | 3,7
average            | 1         | 6
sum                | 3         | 9,10,11
flag               | 2         | 4,8

================================================================================
```

```
4. LITERAL STATISTICS
================================================================================

Literal            | Type    | Frequency | Lines
-------------------|---------|-----------|----------------------
10                 | INTEGER | 3         | 1,5,8
25                 | INTEGER | 2         | 2,6
100                | INTEGER | 1         | 10
3.14               | FLOAT   | 2         | 2,6
2.5                | FLOAT   | 1         | 9
'A'                | CHAR    | 1         | 3
"Hello"            | STRING  | 2         | 4,11
"World"            | STRING  | 1         | 7

================================================================================
```

```
================================================================================
5. TOKEN SUMMARY STATISTICS
================================================================================

Total Tokens: 87
Unique Token Types: 24
Total Lines with Code: 12
Empty/Ignored Lines: 3

Most Frequent Token: IDENTIFIER (15 occurrences, 17.24%)
Least Frequent Token: KEYWORD_ELSE (1 occurrence, 1.15%)

Average Tokens per Line: 7.25
Maximum Tokens in a Single Line: 12 (Line 4)
Minimum Tokens in a Single Line: 4 (Line 10)

================================================================================
```

```
================================================================================
6. TOKEN CATEGORY BREAKDOWN
================================================================================

Category          | Total | Percentage
------------------|-------|-----------
KEYWORD           | 12    | 13.79%
IDENTIFIER        | 15    | 17.24%
LITERAL           | 14    | 16.09%
OPERATOR          | 17    | 19.54%
SEPARATOR         | 29    | 33.33%
COMMENT           | 0     | 0.00%

================================================================================
END OF REPORT
================================================================================
```

Test Program

```
// W++ Token Analyzer Test Program
// This program demonstrates all token types
int main() {
    // Variable declarations
    int age = 25;
    float temperature = 36.6;
    double pi = 3.14159265359;
    char grade = 'A';
    string name = "John Doe";
    // Multiple declarations
    int x = 10, y = 20, z = 30;
    float a = 1.5, b = 2.5, c = 3.5;
    // Arithmetic operations
    int sum = x + y + z;
    float product = a * b * c;
    double division = pi / 2.0;
    int remainder = 100 % 7;
    // Conditional logic
    if (age >= 18 && age <= 60) {
        print "Adult age range";
        string status = "Working";
    } else if (age > 60) {
        print "Senior citizen";
        string status = "Retired";
    } else {
        print "Minor";
        string status = "Student";
    }
    // While loop
    int counter = 0;
    while (counter < 10) {
        counter = counter + 1;
        print "Counter value: ";
        print counter;
    }
    // For loop (if supported)
    for (int i = 0; i < 5; i = i + 1) {
        print "Iteration: ";
        print i;
    }
    // Logical operations
    bool flag = true;
    bool result = (x > y) && (a < b) || !flag;
    // Character operations
    char newline = '\n';
    char tab = '\t';
    char backslash = '\\';
    char quote = '\"';
    // String concatenation
    string greeting = "Hello, " + name + "!";
    string path = "C:\\Program Files\\W++";

    return 0;
}
```